

PRODUCT REQUIREMENTS DOCUMENT

Peblo AI Story Buddy & Quiz Component | Flutter Mobile App

Document Version v1.0	Submission Deadline 25 June 2026
Role Applied For Flutter / Mobile App Developer Intern	Tech Stack Flutter (Dart) — Android & iOS
Company Peblo — India's AI Learning Playground	Target Audience Children aged 5–12, India (mid-range Android)

1. Executive Summary

This document defines the complete product requirements for the Peblo AI Story Buddy & Quiz Component — a single-screen Flutter mobile application built as part of the Peblo Mobile App Developer Internship Challenge.

Peblo is India's AI-powered learning universe for children, blending original animated stories, curriculum-aligned games, interactive quizzes, age-appropriate news, and a personal AI buddy companion. The five mascot worlds — Peblo Buddy (AI companion), Raga (storytelling), Masti (edutainment games), Vidya (adaptive quizzes), and Samachar (news) — each deliver a distinct, joyful learning experience rooted in Indian culture and values.

This feature component sits at the intersection of the Raga (storytelling) and Vidya (quiz) worlds: a child listens to a narrated story snippet and then engages with a data-driven interactive quiz — building comprehension, curiosity, and joy simultaneously.

2. Product Vision & Goals

☐ Vision Statement

Build a delightful, performant, child-first Flutter screen where a young learner can hear a story narrated by their AI buddy, then answer a comprehension quiz — experiencing celebration on success and gentle encouragement on wrong answers — all on a mid-range Android device with no perceptible lag.

2.1 Key Goals

Joy First	Every interaction — button press, audio start, quiz reveal, correct answer — should feel magical and exciting for a child aged 5–12.
Data-Driven	The quiz renderer must be fully generic: it reads from a JSON payload and works for any question, any answer count (3–5), any option text — without code changes.
Performance	Animations at 60fps, cold start < 2s, APK size optimized for mid-range Android (~3GB RAM devices like Redmi, Realme budget series).

Resilient	TTS failure, network timeout, and audio interruptions must be handled gracefully — friendly messages, retry affordance, zero crashes.
Production-Grade	Clean state management (Riverpod), no memory leaks, correct audio lifecycle, proper widget tree structure.

3. Target Audience

3.1 Primary Users

Age Range	5–12 years (early and middle grade)
Region	India — primarily Tier 2 & Tier 3 cities
Device Profile	Mid-range Android: ~3GB RAM, MediaTek/Snapdragon 400-series, 720p screens
Language Context	English UI; culturally Indian content and characters
Usage Context	Supervised or independent learning sessions, 5–15 min bursts

3.2 Secondary Users

- Parents — monitor child engagement, ensure safe & educational content
- Teachers — potentially assign story+quiz pairs as homework

4. Feature Requirements

4.1 Complete Feature List with Priorities

Feature	Priority	Platform	Complexity	Notes
Vibrant child-friendly screen UI	P0	Both	Medium	Peblo brand colors: purple #5C2D91, orange #FF6B35. Rounded cards, large touch targets (min 48dp).
AI Buddy mascot display	P0	Both	Low	Cute character (Pip the Robot concept or Peblo buddy SVG/PNG placeholder). Shows idle, speaking, celebrating states.
Story text card	P0	Both	Low	Displays full story snippet. Scrollable if text grows. Card with shadow, rounded corners.
Read Me a Story button	P0	Both	Low	Large, prominent, animated tap effect. Disabled during playback.
Native TTS narration	P0	Both	Medium	flutter_tts. Speak story text. Handle loading, playing, complete, error states.
TTS loading / preparing state	P0	Both	Low	Spinner + 'Getting ready...' message while TTS engine initialises.
TTS error handling + retry	P0	Both	Medium	Friendly child message ('Oops! My voice got stuck.'). Retry button. No crash, no hang.
JSON-driven quiz reveal	P0	Both	High	Quiz animates in after audio completes. Reads question, options[], answer from JSON. Handles 3/4/5 options.

Wrong answer feedback	P0	Both	Medium	Card shake animation (AnimationController). Haptic feedback (HapticFeedback.mediumImpact). Try-again allowed.
Correct answer celebration	P0	Both	Medium	Confetti animation (confetti package). Buddy switches to happy/celebrating state. 'You got it!' success overlay.
Riverpod state management	P0	Flutter	High	AudioStateNotifier, QuizStateNotifier, BuddyStateNotifier. No unnecessary rebuilds.
ElevenLabs TTS integration	P1	Both	High	Bonus: richer voice. Fallback to flutter_tts if network fails. Cache audio file locally.
Audio caching (remote)	P1	Both	Medium	Cache remote audio with flutter_cache_manager or path_provider + manual cache.
Buddy character animations	P1	Both	Medium	Lottie animations for idle bob, speaking mouth move, celebration dance.
Accessibility basics	P1	Both	Low	Semantics labels on quiz options, sufficient color contrast (WCAG AA), large text mode support.
Performance profiling	P1	Both	Medium	Frame timing in Flutter DevTools. Document before/after widget rebuild counts.
Story re-read capability	P2	Both	Low	After quiz success, allow replaying the story.
Multiple story/quiz pairs	P2	Both	Medium	Swipeable card deck with multiple story+quiz combos.

5. Screen Design & UI Specification

5.1 Design Philosophy

The screen must feel like entering a storybook world — warm, magical, and safe. Design principles for child-first UI:

- Large touch targets: minimum 56dp × 56dp for all interactive elements
- High contrast: text contrast ratio ≥ 4.5:1 (WCAG AA)
- Rounded everything: cards, buttons, avatars — border-radius ≥ 16dp
- Bold, friendly typography: use a rounded/playful font (e.g. Nunito or Fredoka One from Google Fonts)
- Motion with purpose: animations celebrate progress, never distract during reading
- No clutter: one clear primary action visible at all times

5.2 Color System

Primary Purple	#5C2D91 — Peblo brand primary, CTAs, headers
Warm Orange	#FF6B35 — accents, highlight, correct-answer glow
Sky Teal	#00BCD4 — quiz option backgrounds, secondary accents
Golden Yellow	#FFC107 — star bursts, achievement badges, confetti
Off-White BG	#F8F4FF — screen background with subtle purple tint
Success Green	#43A047 — correct answer highlight, success state
Error Red	#E53935 — wrong answer outline (brief shake, then reset)



6. Audio & TTS System

6.1 TTS Architecture

Tier 1 (Required)	flutter_tts — device native engine. Works offline. Zero latency to start. Default voice.
Tier 2 (Bonus)	ElevenLabs API — warm, expressive voice. Requires network. Must cache on first fetch.

6.2 Audio State Machine

6.3 State UI Mapping

Confidential | Peblo Internship Challenge 2026 | Page 4

ERROR	Friendly message card: 'Oops! My voice got a bit shy. Try again?' + Retry button.
COMPLETED	Button hidden. Quiz slides up from bottom. Buddy points to quiz excitedly.

6.4 ElevenLabs Caching Strategy

- On first successful fetch: save audio bytes to app cache directory via `path_provider`
- Key: SHA-256 hash of the story text string — ensures cache invalidates if text changes
- On subsequent taps: check cache first, serve local file instantly, skip network call
- Cache expiry: 7 days (clear on app update or manual cache purge)
- Failure fallback: if ElevenLabs fetch fails (timeout, no network, API error) → silently fall back to `flutter_tts`. Show no error to child.

7. Interactive Quiz Engine

7.1 Data Contract (JSON)

```
{  "question": "What colour was Pip the Robot's lost gear?",  "options": ["Red", "Green", "Blue", "Yellow"],  "answer": "Blue" }
```

The renderer must handle any valid `QuizModel` — it iterates `options.length` dynamically. No hardcoded option count, no hardcoded strings.

7.2 QuizModel (Dart)

```
class QuizModel {  final String question;  final List<String> options;  // 3, 4, or 5 items  final String answer;  const QuizModel({    required this.question,    required this.options,    required this.answer,  });  factory QuizModel.fromJson(Map<String, dynamic> json) => QuizModel(    question: json['question'] as String,    options: List<String>.from(json['options']),    answer: json['answer'] as String,  ); }
```

7.3 Quiz State Machine

HIDDEN	Quiz not visible. Audio not yet complete.
REVEALING	Slide-up + fade-in animation (400ms ease-out). Buddy points excitedly.
AWAITING_ANSWER	All options tappable. None selected yet.
WRONG	Tapped option shakes (300ms). Red outline briefly. <code>HapticFeedback.mediumImpact()</code> . State resets to <code>AWAITING_ANSWER</code> after 600ms.
CORRECT	Tapped option glows green. Confetti fires. Buddy dances. <code>ConfettiController.play()</code> . Show success overlay.

7.4 Quiz Option Rendering

- Options rendered via `ListView.builder(itemCount: quiz.options.length)` or `Wrap` widget
- Each option is an `OptionTile` widget — stateless, receives: `text`, `isSelected`, `isCorrect`, `isWrong`, `onTap`
- Color logic: wrong = red border + shake; correct = green fill; default = white with purple border
- Emoji colors per option: auto-assigned from a color palette list (`index % colors.length`)
- Min height 56dp, full width on narrow screens, 2-column grid on wider screens

8. State Management Architecture (Riverpod)

8.1 Provider Structure

audioStateProvider	StateNotifierProvider<AudioNotifier, AudioState>. Manages: idle/preparing/playing/completed/error. Controls flutter_tts lifecycle.
quizStateProvider	StateNotifierProvider<QuizNotifier, QuizState>. Manages: hidden/revealing/awaiting/wrong/correct. Holds selected option.
buddyStateProvider	StateNotifierProvider<BuddyNotifier, BuddyState>. Manages buddy animation state: idle/loading/speaking/pointing/celebrating/sympathetic.
quizDataProvider	Provider<QuizModel>. Returns the static JSON-parsed quiz. In production: FutureProvider calling backend.

8.2 State Transition Trigger

The key cross-provider transition: when AudioState becomes completed, the AudioNotifier dispatches an event that triggers QuizNotifier to move from hidden to revealing, and BuddyNotifier to move to pointing state. This is handled via a ProviderObserver or a ref.listen() in the widget tree.

```
// In StoryBuddyScreen widget: ref.listen<AudioState>(audioStateProvider, (prev, next) {  if
(next == AudioState.completed) {      ref.read(quizStateProvider.notifier).revealQuiz();
ref.read(buddyStateProvider.notifier).pointAtQuiz();  } });
```

9. Unique & Interesting Feature Ideas

Ideas to Make Peblo's Quiz Stand Out (Beyond the Brief)

9.1 Pip's Mood Reactor

Pip (the buddy mascot) reacts in real-time to audio progress. As the TTS narrates, floating word bubbles pop up above Pip showing key story words — 'robot', 'Whispering Woods', 'gear' — in colorful speech bubbles. This reinforces vocabulary as children listen, bridging audio and visual learning.

9.2 Streak Star System

Above the quiz, a row of 3 empty stars fills as the child answers correctly on the first try. 3-star first-attempt → 'Perfect Score!' banner. 1-2 stars → 'Great Try!' Each star fills with a satisfying pop animation. Stars stored locally (shared_preferences) to show cumulative story stars — encouraging children to re-listen and improve.

9.3 Whisper Hint Mode

If a child taps the same wrong option twice, Pip leans forward and whispers a hint: a small tooltip card slides up showing a clue ('Think about what colour the sky is!'). The hint is part of the quiz JSON ('hint' field, optional). This transforms failure from frustration into guided discovery — very Peblo in spirit.

9.4 Sound-Reactive Story Card

While TTS is playing, the story text card has a subtle animated border that pulses like a sound wave — a soft purple ripple that syncs to a simple animation ticker. This gives tactile visual feedback that narration is happening without requiring precise audio sync, and it's purely Flutter animation — zero performance cost.

9.5 Celebration Personalisation

On correct answer, the confetti colors match the correct option's color (Blue = blue confetti, stars, and sparkles). This micro-personalisation makes the celebration feel connected to the answer — children notice it and feel clever for choosing the right color.

10. Technical Architecture

10.1 Project Structure

```
lib/ ├── main.dart ├── app/ |   ├── peblo_app.dart           # MaterialApp, theme, routing |
features/ |   ├── story_buddy/ |   ├── data/ |   ├── models/ |   ├── quiz_repository.dart |
quiz_model.dart |   ├── repositories/ |   ├── story_buddy_screen.dart |
presentation/ |   ├── screens/ |   ├── story_card.dart |
widgets/ |   ├── buddy_character.dart |   ├── quiz_section.dart |   ├── option_tile.dart |
  ├── read_button.dart |   ├── audio_provider.dart |   ├── quiz_provider.dart |
  ├── providers/ |   ├── buddy_provider.dart |   ├── core/ |   ├── theme/ |   ├── peblo_theme.dart |   ├── services/ |
  ├── tts_service.dart |   ├── audio_cache_service.dart |   ├── assets/ |   ├── animations/ |
# Lottie JSON files |   ├── images/ |   # Buddy character PNGs
```

10.2 Key Dependencies (pubspec.yaml)

flutter_riverpod: ^2.x	State management — AudioNotifier, QuizNotifier, BuddyNotifier
flutter_tts: ^4.x	Native TTS narration on Android (eSpeak/Google TTS) and iOS (AVSpeechSynthesizer)
confetti: ^0.7.x	Celebratory confetti particle animation on correct answer
lottie: ^3.x	Buddy character animation (Lottie JSON for idle/speaking/celebrating states)
just_audio: ^0.9.x	For ElevenLabs audio playback from cached file/URL
path_provider: ^2.x	Get app cache directory for ElevenLabs audio caching
shared_preferences: ^2.x	Persist star streak scores locally between sessions
google_fonts: ^6.x	Nunito / Fredoka One font for child-friendly typography
http: ^1.x	ElevenLabs API calls (bonus tier)
flutter_cache_manager: ^3.x	Robust caching layer for remote audio (optional over manual impl)

11. Performance Strategy

11.1 Target Metrics

Cold start to interactive	< 2 seconds on mid-range Android (Redmi 9A class)
Animation frame rate	≥ 58fps for all animations (confetti, shake, slide-up)
APK size (release)	< 25MB (use --split-per-abi, strip debug symbols)
Memory peak (quiz + confetti)	< 180MB RAM (profile on 3GB device)
TTS first word latency	< 800ms from button tap to first spoken word (native TTS)

11.2 Optimization Techniques

- Use const constructors everywhere possible — prevents unnecessary widget rebuilds
- RepaintBoundary around BuddyCharacter and ConfettiWidget — isolates heavy paint from rest of tree
- Confetti particle count capped at 30 on devices with < 4GB RAM (detect via device_info_plus)
- Lottie animations use LottieBuilder.asset with frameRate: FrameRate.max — let Flutter choose optimal
- No opacity animations on opaque widgets — use FadeTransition with AlwaysStoppedAnimation for static states
- ListView.builder for quiz options (even for 3–5 items) — future-proof and rebuildable
- Audio file read on background isolate (compute()) — never blocks UI thread
- Image assets: WebP format, max 2x density (no 3x on 720p target screens)

12. Error Handling & Edge Cases

TTS engine not available	Show friendly error card. Offer to display story as text only. Log error silently.
ElevenLabs API timeout (> 8s)	Fall back to flutter_tts silently. Child never sees an error.
ElevenLabs API 4xx/5xx	Same fallback. Log to analytics. Do not surface to child.
No internet (ElevenLabs tier)	Check cache first. If no cache and no internet, use flutter_tts. No connectivity error shown.
Audio interrupted (call, headphones pull)	Detect via flutter_tts.setCompletionHandler + onError. Show 'Want to listen again?' state.
App backgrounded during audio	Pause TTS, save state. Resume on foreground. (AppLifecycleListener)
Rapid button tapping	Debounce: ignore taps during PREPARING and PLAYING states via Riverpod state guard.
Quiz JSON malformed	Catch FormatException in fromJson. Show 'Something went wrong' with retry. Never crash.
Low memory	Reduce Lottie frame rate, limit confetti particles, skip image preloading.

13. README Deliverables Checklist

The submission README must address all of the following (as required by Peblo):

1. Why Flutter was chosen (cross-platform, single codebase, excellent animation support, Dart's null-safety, strong community for kids apps)
2. How the audio → quiz transition state works (AudioState.completed triggers QuizNotifier.revealQuiz() via ref.listen)

3. How the quiz is data-driven (QuizModel.fromJson, ListView.builder over options, answer comparison by value not index)
4. Caching approach for remote audio (SHA-256 keyed local file in app cache dir, 7-day TTL, flutter_tts fallback)
5. Audio loading and failure state handling (state machine diagram, error card, retry button)
6. Performance profiling (Flutter DevTools timeline, before/after widget rebuild count, frame timing screenshot)
7. Mid-range Android optimizations (const widgets, RepaintBoundary, reduced confetti count, APK split-per-abi, WebP assets)
8. AI usage transparency (where AI was used, one rejected suggestion with reasoning, what didn't work and how it was resolved)

14. Submission Details

Google Form	https://forms.gle/6re5JGsiUgyGBc5r7
Role to Select	Mobile App Developer
File Naming	mobile_app_developer_challenge_<your_name>
Deadline	25 June 2026
Email	hello@mypeblo.com
WhatsApp	+91 7907482262

14.1 Required Deliverables

- GitHub repository — clean, documented, clear project structure
- README.md — covering all 8 sections listed in Section 13 above
- Screen recording — full flow: audio playing → quiz appearing → wrong answer shake → correct answer celebration
- Flutter DevTools screenshot — frame timing before and after performance optimization

15. Evaluation Criteria

Criteria	What We Look For	Weight
Code Quality	Clean, readable, well-structured, properly documented	20%
UI / UX Delight	Smooth, joyful, genuinely child-appropriate, Peblo brand feel	20%
Audio Handling	Correct state machine, leak-free, all edge cases covered	20%
Data-Driven Quiz	Truly generic renderer — not hardcoded — handles any JSON	20%
Production Mindset	Loading states, error handling, performance on modest devices	20%

16. Official Peblo Character Usage

✔ **Permission Granted by Peblo — hello@mypeblo.com**

This section documents the full character usage plan for this internship submission — covering which characters are used, why they were chosen, where they appear in the app, how they are technically integrated, and the permission obtained from Peblo.

16.1 Permission & Attribution

Permission Status: GRANTED

Peblo's HR team explicitly confirmed via email that Raga and Vidya character assets may be used in this internship challenge submission.

Contact: hello@mypeblo.com | WhatsApp: +91 7907482262

Usage Scope: This internship assessment submission only. Not for commercial redistribution.

All character assets remain the exclusive intellectual property of Peblo. They are credited clearly in the app README and used solely to demonstrate UI integration quality within the scope of this assessment.

16.2 The Five Peblo Characters — Full Overview

Peblo has created five unique mascot characters, each representing a distinct world within their learning universe. Below is the complete character roster:

Character	World	Mode	Focus	Personality & Role
☐ Peblo Buddy	AI Buddy World	AI Buddy	Companion	A personal companion, always curious, always present. Grows alongside the child, remembers preferences, and provides emotional encouragement.
☐ Raga	Peblo TV	Peblo TV	Storytelling	Spins meaningful tales straight from the heart of India. A graceful rabbit in traditional attire who narrates stories rooted in Indian culture and heritage.
☐ Masti	Game World	Game World	Edutainment	Turns the curriculum into lessons and games kids love. A mischievous, energetic monkey who makes learning feel like play through interactive challenges.
☐ Vidya	Quiz World	Quiz World	Mastery	Adapts quizzes to every child, from practice to Olympiad level. A wise owl professor who guides children through knowledge mastery with patience and encouragement.
☐ Samachar	News World	News World	Awareness	Explains world news simply and thoughtfully for children. A well-dressed elephant journalist who keeps young learners aware of the world around them safely.

16.3 Why These Two Characters Were Chosen

Out of the five Peblo characters, this app uses RAGA and VIDYA — and the choice is intentional, not arbitrary:

	🐼 Raga — Story Narrator	🦉 Vidya — Quiz Master
World	Peblo TV / Storytelling World	Quiz World / Mastery World
Why chosen	The challenge is built around story narration — Raga IS Peblo's official storytelling mascot. Using her connects the feature directly to her purpose in the Peblo universe.	The challenge ends with an interactive quiz — Vidya IS Peblo's official quiz and mastery mascot. Her appearance when the quiz reveals is semantically perfect.
Alignment	Her Indian cultural identity fits the story about Pip in the Whispering Woods — a world of wonder and imagination.	Her owl-professor persona sets the right tone for a comprehension question — wise, encouraging, and age-appropriate.
Alternative considered	Peblo Buddy was considered as a single character for both sections, but using two characters creates a natural scene transition — Raga hands off to Vidya — which is more engaging for children.	Masti (the monkey) was considered for the celebration state but Vidya celebrating a correct quiz answer is more authentic to her role as the quiz guide.

16.4 Where Each Character Appears (Screen-by-Screen)

App State	Character Visible	Character State	What the Child Sees
App loads	🐼 Raga	Idle / friendly wave	Raga appears top-center with a gentle bounce animation, waving hello to the child. Story card is visible below her.
Before tapping button	🐼 Raga	Idle bob animation	Raga bobs gently up and down. A speech bubble says 'Tap below to hear my story!' encouraging the child to interact.
Button tapped — preparing TTS	🐼 Raga	Loading / thinking	Raga has a curious thinking expression with a small spinner above her head. Caption: 'Getting my story ready...'
TTS playing — narration active	🐼 Raga	Speaking / mouth animated	Raga's mouth animates as if she is speaking. Sound wave ripples gently around her. The story card glows with a soft pulsing border.
TTS error occurred	🐼 Raga	Shy / embarrassed	Raga looks shy with a small sweat drop. Friendly message: 'Oops! My voice got a bit shy. Try again?' with a retry button.
Audio completed — quiz reveals	🐼 Raga → 🦉 Vidya	Raga waves goodbye, Vidya slides in	Smooth crossfade: Raga waves and shrinks to corner, Vidya slides up from bottom alongside the quiz card. This is the key character handoff moment.
Quiz active — awaiting answer	🦉 Vidya	Pointing at quiz options	Vidya stands beside the quiz card, pointing at the options with an encouraging expression. Her glasses glint with a small sparkle.
Wrong answer tapped	🦉 Vidya	Gentle sympathetic expression	Vidya shakes her head softly with a kind smile. Caption near her: 'Almost! Try again!' The quiz option shakes but Vidya stays calm and supportive.
Correct answer tapped	🦉 Vidya	Full celebration dance	Vidya jumps with joy, stars burst from around her. Confetti rains down. 'You got it! Brilliant!' banner appears. This is the peak emotional moment.
Success state (final)	🦉 Vidya + 🐼 Raga (small)	Both celebrate together	Bonus delight: tiny Raga reappears in corner giving a thumbs up while Vidya dances. Makes the child feel celebrated by the whole Peblo world.

16.5 How Characters Are Technically Integrated

Asset File Structure

```
assets/ └─ images/ └─ raga_idle.png # Raga - default floating state └─
raga_speaking.png # Raga - mouth open, narrating └─ raga_thinking.png # Raga -
loading/preparing state └─ raga_shy.png # Raga - error/embarrassed state └─
raga_wave_bye.png # Raga - handoff to Vidya └─ vidya_idle.png # Vidya -
default quiz state └─ vidya_pointing.png # Vidya - pointing at quiz options └─
vidya_sympathetic.png # Vidya - wrong answer, kind expression └─ vidya_celebrating.png #
Vidya - correct answer, full joy
```

pubspec.yaml Registration

```
flutter: assets: - assets/images/raga_idle.png - assets/images/raga_speaking.png -
assets/images/raga_thinking.png - assets/images/raga_shy.png -
assets/images/raga_wave_bye.png - assets/images/vidya_idle.png -
assets/images/vidya_pointing.png - assets/images/vidya_sympathetic.png -
assets/images/vidya_celebrating.png
```

BuddyAssets Constants Class

```
// lib/core/constants/buddy_assets.dart // Single source of truth - swap assets here without
touching widgets abstract class RagaAssets { static const idle =
'assets/images/raga_idle.png'; static const speaking = 'assets/images/raga_speaking.png';
static const thinking = 'assets/images/raga_thinking.png'; static const shy =
'assets/images/raga_shy.png'; static const waveBye = 'assets/images/raga_wave_bye.png'; }
abstract class VidyaAssets { static const idle = 'assets/images/vidya_idle.png';
static const pointing = 'assets/images/vidya_pointing.png'; static const sympathetic =
'assets/images/vidya_sympathetic.png'; static const celebrating =
'assets/images/vidya_celebrating.png'; }
```

BuddyCharacter Widget — State-Driven Asset Selection

```
// lib/features/story_buddy/presentation/widgets/buddy_character.dart class BuddyCharacter extends
ConsumerWidget { const BuddyCharacter({super.key}); @override Widget build(BuildContext context,
WidgetRef ref) { final audioState = ref.watch(audioStateProvider); final quizState =
ref.watch(quizStateProvider); // Determine which character and which asset to show final String
assetPath = _resolveAsset(audioState, quizState); return RepaintBoundary( // Isolates
repaints from rest of tree child: AnimatedSwitcher( duration: const Duration(milliseconds:
400), transitionBuilder: (child, anim) => FadeTransition( opacity: anim,
child: ScaleTransition(scale: anim, child: child), ), child: Image.asset(
assetPath, key: ValueKey(assetPath), // Triggers AnimatedSwitcher on change
height: 160, fit: BoxFit.contain, ), ); } String
_resolveAsset(AudioState audio, QuizState quiz) { // Quiz states take priority - Vidya owns the quiz
section if (quiz == QuizState.correct) return VidyaAssets.celebrating; if (quiz ==
QuizState.wrong) return VidyaAssets.sympathetic; if (quiz == QuizState.awaitingAnswer ||
quiz == QuizState.revealing) return VidyaAssets.pointing; // Audio states - Raga owns the
story section if (audio == AudioState.playing) return RagaAssets.speaking; if (audio ==
AudioState.preparing) return RagaAssets.thinking; if (audio == AudioState.error)
return RagaAssets.shy; if (audio == AudioState.completed) return RagaAssets.waveBye;
return RagaAssets.idle; // Default: Raga idle on app load }
```

16.6 Character Handoff Animation — Raga to Vidya

The most visually distinctive moment in the app is the character handoff when audio completes and the quiz reveals. This transition must feel magical — not abrupt.

Duration	600ms total — 200ms Raga fade-out + 400ms Vidya slide-in + fade-in overlap
Raga exit	Scale down to 0.6 + fade out + slight leftward slide (dx: -20). She 'steps aside' gracefully.
Vidya entrance	Slides up from y+30 to y=0 with fade-in. Feels like she walked onto the stage.
Sound	Optional: soft 'pop' sound on Vidya appearance (audioplayers package, tiny asset)

Implementation	AnimatedSwitcher with custom transitionBuilder handles this automatically via ValueKey change on assetPath
Fallback	If animation causes jank on low-end device: skip to simple crossfade (400ms FadeTransition only)

16.7 README Credit Block

The following must be included verbatim in the project README.md:

```
## Character Assets & Attribution

The characters Raga (storytelling mascot) and Vidya (quiz mascot) featured in this app are original creations of Peblo (mypeblo.com).

These assets are used with explicit written permission from the Peblo HR team, granted via email (hello@mypeblo.com) for the sole purpose of this Mobile App Developer Internship Challenge submission.

All characters remain the exclusive intellectual property of Peblo. They are not for redistribution, commercial use, or reuse outside this assessment.

> Raga: Peblo TV / Storytelling World | Vidya: Quiz World / Mastery
```

16.8 What Happens If Permission Was NOT Granted

For completeness, here is the documented fallback plan — in case permission had not been received in time:

- Use royalty-free character illustrations from freepik.com (free with attribution) — a bunny storyteller for Raga's role, an owl professor for Vidya's role
- Label all placeholder assets clearly in README: 'Placeholder character — not affiliated with Peblo'
- Structure the BuddyAssets constants class identically — swap real assets in with a single file replacement
- This fallback was never needed — Peblo confirmed permission promptly and enthusiastically

Build something magical. Every child deserves a joyful learning experience.
✧ Good luck — Team Peblo ✧